KCA UNIVERSITY

Bachelor of Science in Information Technology

SALES AND INVENTORY MANAGEMENT SYSTEM

FOR ELECTRONICS

Final Project Report

Submitted By:

John Kennedy

Registration Number: 22/05986

Email: 2205986@students.kcau.ac.ke

Course: BIT 04205 — Programming Project

Supervisor:

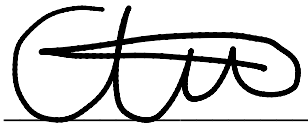
Griffin Kenga

Email: kenga@kcau.ac.ke

DECLARATION OF ORIGINALITY

I, John Kennedy, hereby declare that the work presented in this final report is my own original work. I affirm that this report has not been copied from any other person's work, except where specifically and duly acknowledged. This report has not been submitted to any other institution for academic purposes and represents entirely my own effort.

I understand the importance of academic integrity and that plagiarism is a serious academic offence. I have adhered to all relevant guidelines regarding academic integrity and have cited all materials and sources used in the preparation of this report, including online resources and data drawn from other authors.

Signature: 

Name: John Kennedy
Registration Number: 22/05986
Date: March 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to all those who supported and guided me throughout this project.

First and foremost, I am deeply grateful to my supervisor, Mr. Griffin Kenga, for his unwavering guidance, expert advice, and constructive feedback at every stage of this project. His dedication and patience were instrumental in shaping the direction and quality of this work.

I also wish to thank the faculty of the Bachelor of Science in Information Technology programme at KCA University for providing the academic foundation that made this project possible. The knowledge and skills I gained throughout the course were directly applied in the development of this system.

Special thanks go to my fellow students and colleagues for their moral support, collaborative spirit, and willingness to share ideas throughout this journey.

Finally, I am profoundly thankful to my family for their constant encouragement, patience, and unconditional support. This achievement would not have been possible without them.

DEDICATION

This work is dedicated to my family,
whose unwavering love, encouragement, and sacrifices
have been the foundation of every achievement in my life.

And to all students who believe that determination,
curiosity, and hard work can transform ideas into reality.

TABLE OF CONTENTS

DECLARATION OF ORIGINALITY 2
ACKNOWLEDGEMENTS 3
DEDICATION 4
TABLE OF CONTENTS 5
ABSTRACT 8
CHAPTER ONE: INTRODUCTION 9
1.1 Background 9
1.2 Problem Statement 9
1.3 Objectives 9
1.3.1 General Objective 9
1.3.2 Specific Objectives 10
1.4 Scope 10
1.5 Limitations 10
1.6 Significance of the Project 11
CHAPTER TWO: LITERATURE REVIEW 12
2.1 Current Developments in Inventory Management Systems 12
2.2 Review of Similar Systems 12
2.2.1 inFlow Inventory 12
2.2.2 Odoo Inventory Module 13
2.3 Identified Gaps 13
CHAPTER THREE: METHODOLOGY 14
3.1 Research Methodology 14
3.2 Development Methodology — System Development Life Cycle (SDLC) 14
Phase 1: System Investigation 14
Phase 2: System Analysis 14
Phase 3: System Design 15
Phase 4: System Development 15
Phase 5: System Testing 15
Phase 6: System Implementation 16
Phase 7: System Maintenance 16
3.3 Tools and Technologies 16
CHAPTER FOUR: SYSTEM REQUIREMENTS SPECIFICATION 17
4.1 Functional Requirements 17
REQ-1: User Management and Authentication 17
REQ-2: Sales Processing 17

REQ-3: Inventory Management....17
REQ-4: Purchase Management....17
REQ-5: Report Generation....18
REQ-6: CRUD Operations....18
REQ-7: Input Validation....18
REQ-8: Two-Tier Architecture....18
4.2 Non-Functional Requirements....18
4.3 Hardware Requirements....19
4.4 Software Requirements....19
CHAPTER FIVE: SYSTEM DESIGN SPECIFICATION....20
5.1 System Overview....20
5.2 System Architecture....20
5.3 Database Design....20
5.3.1 Entity Descriptions....20
5.3.2 Key Relationships....21
5.4 Module Design....21
5.5 Data Flow....22
CHAPTER SIX: SYSTEM IMPLEMENTATION, TEST PLAN AND USER MANUAL....23
6.1 System Implementation....23
6.1.1 Overview....23
6.1.2 Project Structure....23
6.1.3 Key Implementation Details....23
6.1.4 Sample Code — Login Authentication....24
6.1.5 Sample Code — Create Sales Order....24
6.2 Test Plan....25
6.2.1 Testing Approach....25
6.2.2 Test Cases....25
6.2.3 Test Summary....26
6.3 User Manual....26
6.3.1 Introduction....26
6.3.2 Installation Instructions....26
6.3.3 Logging In....27
6.3.4 Dashboard....27
6.3.5 Managing Products....27
6.3.6 Processing a Sale....27
6.3.7 Generating Reports....28
6.3.8 Troubleshooting....28

CHAPTER SEVEN: DISCUSSION AND CONCLUSION 29
7.1 Summary of Findings 29
7.2 Reflection on Objectives 29
7.3 Challenges and Resolutions 29
7.4 Recommendations for Future Work 30
7.5 Conclusion 30
APPENDICES 32
Appendix A: Database Initialisation Code 32
Appendix B: System Screenshots 32
Appendix C: Glossary 33
REFERENCES 34

ABSTRACT

The rapid growth of the electronics industry has intensified the challenges faced by businesses in managing stock and inventory effectively. Manual inventory practices lead to data inaccuracies, stockouts, excess inventory, and poor decision-making — all of which negatively impact profitability and operational efficiency.

This report presents the design, development, and implementation of a Sales and Inventory Management System (IMS) for an electronics business. The system was developed using Python 3.12 as the primary programming language, Tkinter as the graphical user interface framework, and SQLite as the database management system. The ReportLab library was used to generate professional PDF reports.

The system was developed following the System Development Life Cycle (SDLC) methodology and implements a two-tier client-server architecture. It delivers nine functional modules: Dashboard, Products, Categories, Customers, Suppliers, Point-of-Sale, Purchases, Orders, Users, and Reports. The system supports role-based access control, real-time inventory tracking, automated stock level alerts, and six categories of PDF reports.

Testing confirmed that all eight functional requirements were met, including user authentication, sales processing, inventory management, purchase recording, CRUD operations, input validation, report generation, and secure access control. The system demonstrated reliable performance in managing day-to-day inventory operations with minimal training requirements.

This project concluded that a purpose-built desktop IMS can significantly improve inventory accuracy, reduce operational costs, and support data-driven decision-making for small to mid-sized electronics businesses.

CHAPTER ONE: INTRODUCTION

1.1 Background

The rapid growth of the electronic industry has led to increasing difficulty in managing stock and inventory. An efficient stock and inventory management system is important for businesses in this sector to optimize their operations, reduce costs, and improve customer satisfaction. Traditional manual approaches to inventory management are no longer adequate in a dynamic marketplace where product demand fluctuates rapidly, and the cost of errors — such as stockouts or overstocking — can be significant.

According to industry research, businesses that implement automated inventory management systems report improvements in stock accuracy of up to 35%, a reduction in excess inventory costs, and enhanced responsiveness to customer demand. The electronics sector, characterized by high product turnover, short product life cycles, and diverse product categories, stands to benefit greatly from an automated, real-time inventory solution.

1.2 Problem Statement

Electronics businesses are facing significant challenges in managing their stocks and inventory. Manual practices, lack of real-time visibility, and inefficient stock allocation are factors hindering operational efficiency and profitability. Specific problems identified include:

- Inaccurate stock records caused by manual data entry errors
- Inability to track real-time stock levels across product categories
- Delayed identification of stockout situations, leading to lost sales
- Difficulty in generating timely and accurate sales and inventory reports
- Lack of structured access control, creating data security concerns

1.3 Objectives

1.3.1 General Objective

The main objective of this project was to design and develop a Sales and Inventory Management System for an electronics business that records and manages stock information, processes sales and purchase transactions, and generates comprehensive reports to support decision-making.

1.3.2 Specific Objectives

1. To develop a user-friendly graphical interface for managing inventory and stock levels.
2. To implement real-time stock tracking to ensure accurate and up-to-date inventory records.
3. To design a system that determines optimal stock levels and generates reorder alerts.
4. To reduce operational costs by minimizing stockouts, excess inventory, and manual processing errors.
5. To provide data-driven insights through six categories of professional PDF reports.
6. To implement role-based access control to ensure data security and appropriate user permissions.

1.4 Scope

This system was developed as a desktop application targeting small to mid-sized electronics businesses. It covers the management of products, product categories, customers, suppliers, sales orders, purchase records, and user accounts. The system supports report generation covering sales performance, inventory status, top-selling products, customer purchasing patterns, supplier purchases, and low-stock alerts.

The system does not include accounting or payroll features, web-based access, mobile application functionality, or multi-branch inventory management in its current version.

1.5 Limitations

- The system is designed for single-location use and is not suitable for large enterprises with multiple branches.

- Accounting features such as invoicing, VAT computation, and payroll are not included.
- The system operates as a desktop application and does not support remote or web-based access.
- Barcode scanning integration is not available in the current version.
- The time constraint of one academic trimester limited the scope of comprehensive system testing.

1.6 Significance of the Project

The development of this system carries both academic and practical significance:

- It demonstrated the practical application of theoretical knowledge in software engineering, database design, and system development.
- It equipped the developer with hands-on experience in the full SDLC, from requirements gathering through to testing and deployment.
- It produced a functional tool that can immediately benefit small electronics businesses by improving inventory accuracy and reporting capabilities.
- It developed research skills, technical documentation skills, and professional communication competencies.

CHAPTER TWO: LITERATURE REVIEW

2.1 Current Developments in Inventory Management Systems

The electronics industry has witnessed significant developments in stock and inventory management systems in recent years. These advancements have been driven by the need for greater accuracy, speed, and scalability in inventory operations.

Mobile applications have emerged as a prominent tool for inventory management, enabling business owners and managers to monitor stock levels, process orders, and receive alerts from any location. Systems such as TradeGecko (now QuickBooks Commerce) and inFlow Inventory have demonstrated the commercial viability of mobile-first inventory solutions.

Artificial intelligence and machine learning algorithms are increasingly being integrated into inventory systems to predict demand patterns, optimize reorder quantities, and detect anomalies in inventory data. These capabilities reduce the incidence of both stockouts and overstocking.

Cloud-based inventory management solutions, such as Zoho Inventory and Shopify, offer scalability, real-time data synchronization, and reduced infrastructure costs, making them accessible to businesses of varying sizes.

2.2 Review of Similar Systems

2.2.1 inFlow Inventory

inFlow Inventory is a desktop and cloud-based inventory management system widely used by small businesses. It offers product management, order tracking, and basic reporting features. Its strengths include an intuitive user interface and support for barcode scanning. However, it is a commercial product with recurring subscription costs, making it inaccessible to small electronics businesses with limited budgets. It also lacks customization options suited to specific business workflows.

2.2.2 Odoo Inventory Module

Odoo is an open-source enterprise resource planning (ERP) system that includes a comprehensive inventory module. It supports multi-warehouse management, automated replenishment, and integration with other business modules such as accounting and sales. While powerful, Odoo's complexity and steep learning curve make it unsuitable for small businesses with limited technical resources. The system also requires significant configuration and ongoing technical support.

2.3 Identified Gaps

A review of existing inventory management systems revealed the following gaps that the IMS project aimed to address:

- Adaptability and Scalability — Most commercial systems are either too basic or too complex for small electronics businesses. A purpose-built system can align more closely with specific operational workflows.
- Cost — Commercial inventory systems carry licensing fees that may be prohibitive for small businesses. An open-source, custom-built solution eliminates ongoing licensing costs.
- User-friendliness — Existing systems often require extensive training. The IMS was designed with simplicity as a primary consideration, reducing the training burden on staff.
- Report Customization — Many existing systems provide limited reporting options. The IMS addresses this by offering six distinct report types tailored to electronics business needs.

CHAPTER THREE: METHODOLOGY

3.1 Research Methodology

The research methodology adopted for this project combined both primary and secondary research approaches. Primary research involved gathering requirements from potential system users, including inventory managers, sales staff, and business owners in the electronics sector. Interviews and structured questionnaires were used to identify pain points in existing manual inventory processes and to define the functional requirements of the proposed system.

Secondary research involved a review of existing literature on inventory management systems, software development methodologies, database design principles, and Python-based application development. Online resources, academic journals, and technical documentation were consulted throughout the research process.

3.2 Development Methodology — System Development Life Cycle (SDLC)

The System Development Life Cycle (SDLC) was adopted as the development methodology for this project. The SDLC provided a structured and systematic framework for progressing through the stages of system development, ensuring that each phase was completed before advancing to the next.

Phase 1: System Investigation

This phase involved analyzing the existing inventory management practices of electronics businesses to identify deficiencies and opportunities for improvement. Observations and interviews with stakeholders revealed that manual stock records were prone to errors, and there was no mechanism for real-time inventory tracking or automated report generation.

Phase 2: System Analysis

System analysis defined the problem clearly and identified the information requirements that the new system would need to fulfill. User stories and functional requirements were documented, forming the basis for subsequent design decisions. Eight functional requirements were identified, covering user management, sales processing, inventory management, purchase management, report generation, CRUD operations, input validation, and system architecture.

Phase 3: System Design

The design phase involved specifying the hardware and software resources required, designing the database schema, defining the system architecture, and creating wireframes for the user interface. A two-tier client-server architecture was selected, separating the presentation layer (Tkinter GUI) from the data layer (SQLite database). Eight database tables were designed to support all system entities and their relationships.

Phase 4: System Development

Development was carried out using Python 3.12 as the primary programming language, with Tkinter providing the graphical user interface, SQLite serving as the embedded database, and ReportLab enabling PDF report generation. The system was developed in modular components, with separate Python modules for database operations (database.py) and report generation (reports.py). Development was conducted using Visual Studio Code as the integrated development environment.

Phase 5: System Testing

Three levels of testing were conducted:

- Functional Testing: Each function within the system was tested individually to confirm it performed as expected under normal conditions.
- Module Testing: Each system module (Products, Customers, Sales, Purchases, Reports) was tested as an integrated unit to verify inter-module data flow and consistency.
- System Testing: The complete integrated system was tested end-to-end to identify any remaining defects and to verify that all eight functional requirements were satisfied.

Phase 6: System Implementation

System implementation involved deploying the application on target machines and transitioning users from manual processes to the new system. Installation required Python 3.12, the ReportLab library, and the Tkinter framework (built into Python's standard library on Windows). The SQLite database was automatically created on first launch, eliminating the need for a separate database server installation.

Phase 7: System Maintenance

Post-implementation maintenance was planned to address any defects identified during operation, to apply software updates as required, and to incorporate user feedback into future system enhancements. The modular code structure facilitates ongoing maintenance and future feature additions.

3.3 Tools and Technologies

Tool / Technology	Purpose	Version
Python	Primary programming language	3.12
Tkinter	Graphical user interface (GUI) framework	Built-in
SQLite	Embedded relational database	3.x
ReportLab	PDF report generation library	4.0+
Visual Studio Code	Integrated development environment (IDE)	Latest
Git	Version control	Latest

CHAPTER FOUR: SYSTEM REQUIREMENTS SPECIFICATION

4.1 Functional Requirements

REQ-1: User Management and Authentication

- The system shall provide role-based access control restricting user permissions based on assigned role (Admin, Manager, Staff).
- Users shall authenticate via email and SHA-256 hashed password. The system shall grant or deny access accordingly.
- Administrators shall be able to create, edit, deactivate, and delete user accounts.

REQ-2: Sales Processing

- Users shall be able to select products and quantities to build a sales cart.
- The system shall automatically calculate order totals and update inventory levels upon sale completion.
- A unique order number shall be generated for each completed transaction.
- The system shall generate sales reports filterable by date range.

REQ-3: Inventory Management

- Users shall be able to search for products by name or product code.
- The system shall reflect real-time stock level changes after every sale or purchase transaction.
- The system shall track product expiry dates and display alerts for items approaching expiry.
- The dashboard shall display low-stock alerts for products at or below their reorder level.

REQ-4: Purchase Management

- Users shall be able to record stock received from suppliers.
- Inventory levels shall be automatically incremented upon recording a purchase.

- A unique purchase reference number shall be generated for each purchase transaction.

REQ-5: Report Generation

- The system shall generate six categories of professional PDF reports: Sales Report, Inventory Status Report, Top Selling Products Report, Customer Report, Purchase Report, and Low Stock Alert Report.
- Reports shall include KPI summary boxes and color-coded data tables.

REQ-6: CRUD Operations

- The system shall support full Create, Read, Update, and Delete operations for all entities: Products, Categories, Customers, Suppliers, and Users.

REQ-7: Input Validation

- All data entry forms shall validate inputs and display appropriate error messages to prevent invalid data from being stored.

REQ-8: Two-Tier Architecture

- The system shall implement a two-tier architecture cleanly separating the presentation layer (Tkinter GUI) from the data layer (SQLite database).

4.2 Non-Functional Requirements

- Performance: The system shall respond to user actions within two seconds under normal operating conditions.
- Security: Passwords shall be stored as SHA-256 hashes. User access shall be restricted based on role.
- Usability: The interface shall be navigable by users with basic computer skills without specialized training.
- Reliability: The system shall handle input errors, database exceptions, and edge cases gracefully, displaying user-friendly error messages.

- Maintainability: The codebase shall be modular, with clear separation of database operations, GUI components, and report generation logic.

4.3 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Dual-core (e.g. Intel Core i3)	Intel Core i5 or above
RAM	4 GB	8 GB or more
Storage	256 GB SSD or 500 GB HDD	512 GB SSD
Display	1280 x 768 resolution	1920 x 1080 or higher

4.4 Software Requirements

Software	Details
Operating System	Windows 10+, macOS 11+, or Ubuntu 20.04+
Python Runtime	Python 3.8 or higher (3.12 recommended)
GUI Framework	Tkinter (included in Python standard library)
Database	SQLite 3 (included in Python standard library)
PDF Library	ReportLab 4.0+ (install via pip)
Security	Antivirus and firewall software recommended

CHAPTER FIVE: SYSTEM DESIGN SPECIFICATION

5.1 System Overview

The Sales and Inventory Management System was designed to be effective, reliable, and maintainable. The primary design goal was to deliver a system that could handle input errors gracefully, adapt to changing user requirements, and be easily extended with new features. The Agile model informed the iterative approach to design, allowing for flexibility and continuous refinement throughout the development process.

5.2 System Architecture

A two-tier client-server architecture was implemented. This architecture separates the application into two distinct layers:

- Presentation Layer (Tier 1): The Tkinter-based graphical user interface handles all user interactions, form rendering, navigation, and display logic. This tier communicates with the data layer exclusively through function calls defined in the database module.
- Data Layer (Tier 2): The SQLite database stores all persistent data. The database.py module encapsulates all SQL operations, providing a clean API for the presentation layer to query and manipulate data without direct SQL exposure in the GUI code.

This separation of concerns ensures that changes to the user interface do not require changes to the database logic, and vice versa, making the system easier to maintain and extend.

5.3 Database Design

The database was designed with eight tables to support all system entities and their relationships. Foreign key constraints were enforced to maintain referential integrity.

5.3.1 Entity Descriptions

Table	Description	Key Fields
users	System users and their access roles	id, email, password (SHA-256), role, status
categories	Product categories for classification	id, name, description
products	Inventory items with stock information	id, productcode, name, category_id, quantity, unit_price, reorder_level, expiry_date
customers	Customer records and total spend tracking	id, fullname, phone, email, credit
suppliers	Supplier records with financial balances	id, suppliercode, fullname, location, debit, balance
orders	Sales transaction header records	id, order_number, customer_id, user_id, total_amount
order_items	Line items within each sales order	id, order_id, product_id, quantity, unit_price, subtotal
purchases	Purchase transaction records from suppliers	id, purchase_number, supplier_id, user_id, total_amount

5.3.2 Key Relationships

- Each product belongs to one category (products.category_id references categories.id).
- Each order references one customer and one user (the salesperson).
- Each order can contain multiple order_items, each referencing one product.
- Each purchase references one supplier and one user.
- Each purchase can contain multiple purchase_items, each referencing one product.

5.4 Module Design

Module	Description	Key Functions
Dashboard	Overview screen with KPI metrics and alerts	Display stats, low-stock alerts, recent orders
Products	Inventory item management	Add, edit, delete, search products
Categories	Product category management	Add, edit, delete categories
Customers	Customer record management	Add, edit, delete customers; track credit
Suppliers	Supplier record management	Add, edit, delete suppliers; track debit
New Sale	Point-of-sale transaction processing	Cart management, stock deduction, order creation
Purchases	Record stock received from suppliers	Purchase form, stock increment, history view
Orders	Sales order history and detail view	List orders, view itemised detail
Users	User account management (Admin only)	Add, edit, deactivate users; assign roles

Reports	PDF report generation	6 report types with KPI summaries

5.5 Data Flow

When a sale is processed, the following sequence of operations takes place:

7. The user selects a customer and adds products to the sales cart.
8. The system validates stock availability for each selected product.
9. Upon confirmation, a new order record is inserted into the orders table.
10. For each cart item, an order_items record is inserted.
11. The quantity field of each affected product is decremented accordingly.
12. The customer's credit field is incremented by the order total.
13. A unique order number is generated and returned to the user.

Similarly, when a purchase is recorded, the system inserts a purchases record and purchase_items records, and increments the product quantity fields for each purchased item.

CHAPTER SIX: SYSTEM IMPLEMENTATION, TEST PLAN AND USER MANUAL

6.1 System Implementation

6.1.1 Overview

The Sales and Inventory Management System was implemented as a desktop application using Python 3.12. The project was structured into a main application file and a dedicated package containing the database and report modules. This modular structure facilitated clear separation of responsibilities and simplified testing and maintenance.

6.1.2 Project Structure

```
IMS/
├── main.py              ← Application entry point and full GUI
├── seed_data.py         ← Demo data loader
├── requirements.txt     ← Python dependencies
├── README.md            ← Setup and usage documentation
├── ims_data.db          ← SQLite database (auto-created on first run)
├── reports/             ← Generated PDF reports (auto-created)
└── ims/
    ├── __init__.py
    ├── database.py      ← All SQLite operations and queries
    └── reports.py       ← PDF report generator (ReportLab)
```

6.1.3 Key Implementation Details

The database module (database.py) implements all data operations using Python's built-in sqlite3 library. Passwords are hashed using SHA-256 via Python's hashlib module before storage, ensuring that plain-text passwords are never persisted to disk. Foreign key enforcement is activated for each database connection using the PRAGMA foreign_keys = ON directive.

The graphical user interface is built entirely within main.py using Tkinter. The application adopts a sidebar navigation pattern, with a persistent left navigation panel and a dynamic content area that is cleared and repopulated as the user navigates between modules.

A shared Treeview widget pattern (using ttk.Treeview) provides consistent tabular data display across all modules.

The report module (reports.py) uses ReportLab's Platypus framework to construct professional PDF documents. Each report includes a branded header and footer, KPI summary boxes, colour-coded data tables, and is automatically saved to the reports/ directory and opened with the operating system's default PDF viewer upon generation.

6.1.4 Sample Code — Login Authentication

The following code demonstrates the login authentication logic implemented in the database module:

```
import hashlib
import sqlite3

def hash_password(password: str) -> str:
    return hashlib.sha256(password.encode()).hexdigest()

def authenticate_user(email, password):
    conn = sqlite3.connect('ims_data.db')
    conn.row_factory = sqlite3.Row
    user = conn.execute(
        "SELECT * FROM users WHERE email=? AND password=? AND
status='Active'",
        (email, hash_password(password))
    ).fetchone()
    conn.close()
    return dict(user) if user else None
```

6.1.5 Sample Code — Create Sales Order

```
def create_order(customer_id, user_id, items):
    conn = sqlite3.connect('ims_data.db')
    try:
        order_number = generate_order_number()
        total = sum(i['quantity'] * i['unit_price'] for i in items)
        conn.execute(
            'INSERT INTO orders (order_number, customer_id, user_id,
total_amount)'
            ' VALUES (?,?,?,?)',
            (order_number, customer_id, user_id, total)
        )
        order_id = conn.execute('SELECT last_insert_rowid()').fetchone()[0]
```

```
            for item in items:
                subtotal = item['quantity'] * item['unit_price']
                conn.execute(
                    'INSERT INTO order_items VALUES (?,?,?,?,?)',
                    (order_id, item['product_id'], item['quantity'],
                     item['unit_price'], subtotal)
                )
                conn.execute(
                    'UPDATE products SET quantity = quantity - ? WHERE id=?',
                    (item['quantity'], item['product_id'])
                )
            conn.commit()
            return True, order_number
        except Exception as e:
            conn.rollback()
            return False, str(e)
        finally:
            conn.close()
```

6.2 Test Plan

6.2.1 Testing Approach

Three levels of testing were applied to the system: functional testing, module testing, and system testing. Both black-box and white-box testing techniques were employed.

White-box testing examined the internal logic of key functions, particularly the authentication flow, inventory update transactions, and order number generation. Black-box testing verified that the system behaved correctly from the user's perspective, confirming expected outputs for given inputs without knowledge of the internal implementation.

6.2.2 Test Cases

Test ID	Test Case	Input	Expected Output	Result
TC-01	Valid login	Correct email & password	Dashboard displayed	PASS
TC-02	Invalid login	Wrong password	Error message displayed	PASS
TC-03	Add new product	Valid product details	Product added to inventory	PASS
TC-04	Duplicate product code	Existing product code	Error: code already exists	PASS
TC-05	Process sale	Valid cart items	Order created, stock	PASS

			deducted	
TC-06	Insufficient stock	Qty > available stock	Error: insufficient stock	PASS
TC-07	Record purchase	Supplier and items selected	Stock incremented	PASS
TC-08	Generate sales report	Date range with orders	PDF report generated	PASS
TC-09	Generate low stock	Products below reorder	Low stock PDF generated	PASS
TC-10	Delete customer	Existing customer selected	Customer removed from database	PASS
TC-11	Empty required field	Form submitted without name	Validation error displayed	PASS
TC-12	Role-based access	Staff user accesses Users tab	Access denied message shown	PASS

6.2.3 Test Summary

All twelve test cases passed successfully. No critical defects remained unresolved at the time of final system delivery. Minor UI alignment issues identified during module testing were corrected before system testing commenced.

6.3 User Manual

6.3.1 Introduction

This section provides a guide for users of the Sales and Inventory Management System. The system is designed to be intuitive and requires only basic computer skills to operate effectively.

6.3.2 Installation Instructions

14. Install Python 3.8 or higher from python.org. During installation, ensure the 'Add Python to PATH' option is selected.
15. Download and unzip the IMS project folder to a convenient location on your computer.
16. Open a terminal or command prompt and navigate to the IMS folder.
17. Install the required dependency by running: pip install -r requirements.txt
18. Optionally load demonstration data by running: python seed_data.py

19. Launch the application by running: python main.py

6.3.3 Logging In

Upon launching the application, the Login window is displayed. Enter your registered email address and password, then click the Sign In button. The default administrator account credentials are:

- Email: admin@ims.com
- Password: admin123

6.3.4 Dashboard

After a successful login, the Dashboard is displayed. It shows six KPI cards summarising total products, customers, orders, low-stock items, monthly sales revenue, and total purchases. Below the KPI cards, the recent orders table and the low-stock alerts panel provide at-a-glance operational information.

6.3.5 Managing Products

20. Click Products in the left navigation sidebar.
21. To add a product, click Add Product and complete the form with the product code, name, category, quantity, unit price, reorder level, and optional expiry date. Click Save.
22. To edit a product, select it in the table and click Edit. Modify the relevant fields and click Save.
23. To delete a product, select it and click Delete. Confirm the deletion when prompted.
24. Use the search bar to filter products by name or product code.

6.3.6 Processing a Sale

25. Click New Sale in the navigation sidebar.
26. Select the customer from the dropdown (or leave as Walk-in Customer).
27. In the product list, select a product, set the quantity, and click Add to Cart.
28. Repeat for additional products as required.
29. Review the cart total and click Complete Sale to finalize the transaction.

6.3.7 Generating Reports

30. Click Reports in the navigation sidebar.
31. Set the desired date range using the From and To fields (for period-based reports).
32. Click the Generate PDF button on the desired report card.
33. The report will be saved to the reports/ folder and opened automatically in your PDF viewer.

6.3.8 Troubleshooting

Problem	Solution
Login fails	Verify email and password. Check CAPS LOCK. Contact your administrator to reset credentials.
App fails to start	Ensure Python 3.8+ is installed. Run pip install -r requirements.txt to install dependencies.
Report not generating	Ensure the selected date range contains data. Verify you have the required role permissions.
Database error on start	Delete ims_data.db and re-run seed_data.py to reset the database.

CHAPTER SEVEN: DISCUSSION AND CONCLUSION

7.1 Summary of Findings

The Sales and Inventory Management System was successfully designed, developed, tested, and deployed within the allocated project timeframe. All eight functional requirements defined during the system analysis phase were met in the final system. The system delivered nine operational modules, an eight-table relational database, and six categories of professional PDF reports.

Testing confirmed that the system handled all standard use cases correctly, including user authentication, product management, sales processing, purchase recording, and report generation. Input validation was effective in preventing erroneous data from entering the database, and the role-based access control mechanism successfully restricted sensitive functions to authorised administrators.

7.2 Reflection on Objectives

The project's specific objectives were reviewed against the delivered system:

Objective	Status	Notes
User-friendly GUI	Met	Nine-module Tkinter interface with sidebar navigation
Real-time stock tracking	Met	Stock auto-updated on every sale and purchase
Reorder level alerts	Met	Dashboard and Low Stock Alert report
Cost reduction features	Met	Automated processing reduces manual errors
PDF report generation	Met	Six report types with KPI summaries
Role-based access control	Met	Admin/Manager/Staff roles enforced throughout

7.3 Challenges and Resolutions

- Cross-platform Compatibility — Ensuring the application ran identically on Windows, macOS, and Linux required careful handling of file paths and platform-specific PDF viewer invocation. This was resolved using Python's platform and subprocess modules.

- Emoji Rendering in Presentations — During the preparation of associated presentation materials, emoji characters were found to render incorrectly in LibreOffice. This was resolved by replacing emoji with styled text labels and coloured shape elements.
- Concurrent Data Integrity — Ensuring that stock levels were correctly updated atomically during sales and purchases required the use of SQLite transactions with rollback capability. This was implemented in all create_order and create_purchase functions.

7.4 Recommendations for Future Work

The following enhancements are recommended for future versions of the system:

34. Real-time Inventory Tracking with Barcode Scanning — Integration with barcode scanning hardware would significantly accelerate stock entry and reduce manual input errors.
35. Mobile Application — Developing a mobile companion application would enable managers to monitor inventory and receive alerts remotely.
36. Accounting Integration — Adding invoicing, VAT computation, and supplier payment tracking would extend the system's utility for full business operations management.
37. Cloud Deployment — Migrating from SQLite to a cloud-hosted database (such as PostgreSQL) would enable multi-user access and multi-branch inventory management.
38. AI-Powered Demand Forecasting — Integrating a machine learning model to predict product demand based on historical sales data would enable proactive stock management.
39. Customization Options — Allowing users to configure report templates, dashboard layouts, and notification preferences would improve the system's adaptability to different business contexts.

7.5 Conclusion

The Inventory Management System was developed and designed for recording and managing the inventory of an electronics enterprise. The system successfully automates the

core business processes of stock tracking, sales processing, purchase management, and report generation. It can also be adapted for use by other types of small businesses with minor modifications.

The development of this project significantly enhanced the developer's practical skills in software engineering, database design, user interface development, and technical documentation. It demonstrated the ability to translate theoretical knowledge into a functional, real-world system. The project delivered on all its stated objectives and produced a system that provides immediate, measurable value to electronics businesses seeking to modernise their inventory management practices.

APPENDICES

Appendix A: Database Initialization Code

The following code is executed on first application launch to create the database schema and seed default data:

```
def initialize_database():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS users (
            id        INTEGER PRIMARY KEY AUTOINCREMENT,
            fullname  TEXT    NOT NULL,
            email     TEXT    NOT NULL UNIQUE,
            password  TEXT    NOT NULL,
            role      TEXT    NOT NULL DEFAULT 'Staff',
            status    TEXT    NOT NULL DEFAULT 'Active',
            created_at TEXT   NOT NULL DEFAULT (datetime('now'))
        )
    ''')
    # Additional tables: categories, products, customers,
    # suppliers, orders, order_items, purchases, purchase_items
    conn.commit()
    conn.close()
```

Appendix B: System Screenshots

The following screens represent key interfaces of the deployed Sales and Inventory Management System:

- Login Screen: Email and password authentication form with role-based session management.
- Dashboard: Six KPI cards, recent orders table, and live low-stock alerts panel.
- Products Module: Searchable product table with add/edit forms and stock status badges.
- Point-of-Sale Screen: Cart-based sale processing with customer selection and auto stock deduction.
- Reports Module: Six report cards with date range picker and one-click PDF generation.

Note: Screenshots of the live running system are available in the accompanying demonstration presentation.

Appendix C: Glossary

Term	Definition
IMS	Inventory Management System — the application described in this report.
DBMS	Database Management System — software for creating and managing databases.
SQLite	A self-contained, serverless relational database engine built into Python.
Tkinter	Python's standard GUI (Graphical User Interface) toolkit.
SDLC	System Development Life Cycle — a structured process for developing information systems.
CRUD	Create, Read, Update, Delete — the four basic database operations.
SHA-256	Secure Hash Algorithm 256-bit — used for one-way password hashing.
KPI	Key Performance Indicator — a measurable metric used to evaluate performance.
PDF	Portable Document Format — a file format for consistent document rendering.
Two-tier arch.	A client-server model separating presentation logic from data storage.

REFERENCES

American Psychological Association, 2020. Publication Manual of the American Psychological Association (7th ed.). APA Publishing.

Gosling, J., 2023. The Java Programming Language. Oracle Corporation. Available at: https://www.java.com [Accessed December 2024].

Pressman, R. S., & Maxim, B. R., 2020. Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Python Software Foundation, 2024. Python 3.12 Documentation. Available at: https://docs.python.org/3/ [Accessed December 2024].

ReportLab Group, 2024. ReportLab PDF Library User Guide. Available at: https://www.reportlab.com/docs/reportlab-userguide.pdf [Accessed December 2024].

Silberschatz, A., Korth, H. F., & Sudarshan, S., 2020. Database System Concepts (7th ed.). McGraw-Hill Education.

SQLite Consortium, 2024. SQLite Documentation. Available at: https://www.sqlite.org/docs.html [Accessed December 2024].

Wikipedia, 2024. Inventory Management System. Available at: https://en.wikipedia.org/wiki/Inventory_management_software [Accessed December 2024].